Studio Nx

JS 301 — Sampler 음원

2026-10-20

JavaScript 301 — Sampler 음원 (`audio/sampler.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/sampler.js`
작업	S1-11c Sampler MIDI 연주: MIDI 노트 하나를 샘플로 울리는 그래프와 계산 함수 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 300 내보내기 창, 다음 302 Sampler 연결

1. 하는 일

Sampler 악기에 붙은 샘플을 MIDI 노트의 음높이에 맞춰 울립니다. Synth의 `playSynthNote`(291)와 같은 위치에서 같은 모양으로 쓰입니다. 입력은 노트(음높이, 세기, 시작 시각, 길이)와 디코딩된 샘플이고, 출력은 소리를 앞당겨 끊을 때 쓰는 `{ stop }`입니다.

모드가 둘입니다.

모드	소리 나는 방식	무시하는 값
`classic`	건반을 눌러 음높이를 바꿔 치는 방식입니다. 노트 길이만큼 울리고 Release로 끝납니다. Loop를 켜면 샘플을 되풀이합니다.	없음
`oneshot`	드럼처럼 한 번 치면 샘플 전체가 울립니다.	노트 길이, Loop, Sustain, Release

```
playSamplerNote(ctx, dest, p, buffer, pitch, velocity, when, duration)
  buffer 없음 —▶ 아무것도 만들지 않고 빈 { stop } 을 돌려줍니다
  AudioBufferSourceNode(샘플, playbackRate) —▶ GainNode(앰프 엔벨로프) —▶ dest(악기 층)
    classic : Attack → Decay → Sustain, 노트 끝에서 Release, Loop 가 켜져 있으면 샘플을 되풀이
    oneshot : Attack, Decay 만 적용하고 샘플이 끝날 때까지 울림
```

2. 언제 쓰나

쓰는 곳	하는 일
`InstrumentRack.playNote` (302)	트랙의 악기 층이 Sampler이면 이 함수를 불러 노트를 울립니다.
`session.js`, `offline.js` (302)	`samplerSampleIds`로 미리 디코딩할 샘플 번호를 모읍니다.
시험 `sampler_unit.mjs`, `render_sampler.mjs`	계산과 노드 연결, 실제 소리를 확인합니다.

3. 코드

3.1 순수 함수

```
export function samplerMode(value) {
  return value === 'oneshot' ? 'oneshot' : 'classic';
}
```

```
export function samplerRate(pitch, root, tune) {
  return Math.pow(2, (Number(pitch) - Number(root) + (Number(tune) || 0)) / 12);
}
```

```
export function samplerSampleIds(getInstruments, trackIds) {
  const out = [];
  (trackIds || []).forEach(function (trackId) {
    (getInstruments(trackId) || []).forEach(function (inst) {
      if (inst.plugin_code !== 'sampler' || inst.is_enabled === false) return;
      const params = inst.params || {};
      const id = params.sample_id;
      if (id == null || id === '') return;
      const n = Number(id);
      if (isFinite(n) && out.indexOf(n) < 0) out.push(n);
    });
  });
  return out;
}
```

3.2 노트 재생

```
export function playSamplerNote(ctx, dest, p, buffer, pitch, velocity, when, duration) {
  if (!buffer) return { stop: function () {} };
  const mode = samplerMode(p.mode);
  const rate = samplerRate(pitch, p.root_note != null ? p.root_note : 60, p.tune);
  const peak = Math.max(0, Number(p.gain) || 0) * Math.max(0, Math.min(127, velocity)) / 127;
  const a = Math.max(0, Number(p.attack) || 0);
  const d = Math.max(0, Number(p.decay) || 0);

  const src = ctx.createBufferSource();
  src.buffer = buffer;
  src.playbackRate.value = rate;

  const amp = ctx.createGain();
  src.connect(amp);
  amp.connect(dest);

  let end;
  amp.gain.setValueAtTime(0, when);
  if (mode === 'oneshot') {
    // 노트 길이를 무시하고 샘플 전체를 울림. Attack 은 앞을 부드럽게 열고, Decay 가 있으면 그 시간 동안 0
까지 줄임 (드럼의 꼬리 조절)
    if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
    else amp.gain.setValueAtTime(peak, when);
    if (d > 0) amp.gain.linearRampToValueAtTime(0, when + a + d);
    end = when + buffer.duration / rate + 0.02;
    src.start(when);
    src.stop(end);
  } else {
    const s = p.sustain != null ? Number(p.sustain) : 1;
    const r = Math.max(0.005, Number(p.release) || 0);
    const dur = Math.max(0.01, duration);
    if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
    else amp.gain.setValueAtTime(peak, when);
    if (d > 0 && dur > a) amp.gain.linearRampToValueAtTime(peak * s, when + Math.min(dur, a + d));
```

```
    amp.gain.setValueAtTime(envelopeLevel(dur, a, d, s, peak), when + dur);
    amp.gain.linearRampToValueAtTime(0, when + dur + r);
    end = when + dur + r + 0.02;
    if (p.loop === true) {
      src.loop = true;
      src.loopStart = 0;
      src.loopEnd = buffer.duration;
    }
    src.start(when);
    src.stop(end);
  }

  return {
    // 정지 · 루프 되감기 때 소리를 끊음 (뚝 끊기는 소리를 줄이려고 아주 짧은 릴리스를 줌)
    stop: function (at) {
      const t = Math.max(at, ctx.currentTime);
      try { amp.gain.cancelScheduledValues(t); amp.gain.setTargetAtTime(0, t, 0.01); } catch (e) { /* 무
시 */ }
      try { src.stop(t + 0.08); } catch (e) { /* 이미 끝남 */ }
    },
  };
}
```

4. 재생 속도 계산

눌린 건반이 Root(샘플이 원래 내는 음)에서 몇 반음 떨어졌는지에 Tune을 더해서 2의 거듭제곱으로 바꿉니다. 한 옥타브가 12반음이고 주파수가 2배가 되므로, 반음 하나는 2의 12제곱근(약 1.0595)배입니다.

눌린 건반	Root	Tune	재생 속도	들리는 소리
60	60	0	1배	샘플 그대로
72	60	0	2배	한 옥타브 위, 길이는 절반
48	60	0	0.5배	한 옥타브 아래, 길이는 두 배
60	60	+12	2배	Tune으로 한 옥타브 위
60	48	0	2배	Root를 낮추면 같은 건반이 더 높게 울림

`playbackRate`는 음높이와 속도를 함께 바꾸므로, 높은 음일수록 샘플이 빨리 끝납니다. 오래된 하드웨어 샘플러와 같은 동작이고, 드럼이나 짧은 소리에는 이 방식이 가장 자연스럽습니다.

5. 최신 문법으로 쓰면

```
// samplerSampleIds 를 최신 문법으로 쓴 모양
export const samplerSampleIds = (getInstruments, trackIds = []) => [...new Set(
  trackIds
    .flatMap((id) => getInstruments(id) ?? [])
    .filter((inst) => inst.plugin_code === 'sampler' && inst.is_enabled !== false)
    .map((inst) => inst.params?.sample_id)
    .filter((id) => id != null && id !== '')
    .map(Number)
    .filter(Number.isFinite),
)];
```

```
// 음량 계산
const peak = Math.max(0, p.gain ?? 0) * Math.min(127, Math.max(0, velocity)) / 127;
```

전통 문법 (실제 코드)	최신 문법
`out.indexOf(n) < 0`로 중복을 걸러 `push`	`new Set`과 전개 `[...set]`
`forEach` 안에서 `forEach`	`flatMap`
`(getInstruments(trackId) || [])`	`getInstruments(trackId) ?? []`
`params.sample_id` 앞에 `inst.params || {}`	`inst.params?.sample_id`

6. 왜 이렇게

선택	이유
샘플이 없으면 빈 `{ stop }`을 돌려줍니다	샘플을 아직 고르지 않았거나 불러오지 못한 악기도 노트 재생이 멈추지 않아야 합니다. 호출하는 쪽이 `stop`을 항상 부를 수 있게 같은 모양을 지킵니다.
`oneshot`은 샘플 길이를 재생 속도로 나눠 끝 시각을 정합니다	속도가 2배이면 샘플이 절반 시간에 끝납니다. 길이를 그대로 쓰면 `stop` 시각이 너무 늦어서 쓸데없이 노드가 남습니다.
`oneshot`에서 Attack과 Decay는 남겨 둡니다	Attack은 앞의 클릭 소리를 줄이고, Decay가 있으면 그 시간에 걸쳐 0까지 줄여 드럼의 꼬리를 짧게 만들 수 있습니다. Sustain과 Release는 노트 길이에 의존하는 값이라 무시합니다.
`classic`의 엔벨로프는 `envelopeLevel`(291)을 다시 씁니다	노트가 Attack이나 Decay 도중에 끝나도 Release가 그 자리의 값에서 시작해야 소리가 튀지 않습니다. Synth와 같은 계산이므로 같은 함수를 씁니다.
같은 샘플을 연속으로 쳐도 앞의 소리를 끊지 않습니다	노트마다 `AudioBufferSourceNode`를 새로 만들기 때문에 자연스럽게 겹쳐 울립니다. 하이햇을 빠르게 칠 때처럼 앞의 소리의 꼬리가 남는 것이 실제 장비와 같은 소리입니다. 결정 문서(`Default_Instruments_Effects.md`)에서 정한 동작입니다.
계산을 순수 함수로 따로 둡니다	`samplerRate`와 `samplerSampleIds`는 소리를 쓰지 않으므로 Node에서 바로 시험할 수 있습니다.
재생과 내보내기가 같은 함수를 씁니다	`AudioBufferSourceNode`는 `OfflineAudioContext`에서도 똑같이 동작하므로, 내보낸 WAV에도 Sampler 소리가 자동으로 들어갑니다.

7. 시험

- `docs/handoff/s1-11c_tests/sampler_unit.mjs` (39건)는 Node에서 순수 함수와 노드 연결을 확인합니다.
  - 모드 이름: 알 수 없는 값은 `classic`입니다.
  - 재생 속도: 같은 음 1배, 한 옥타브 위 2배, 아래 0.5배, 7반음 위 약 1.4983, Tune ±12, Root 변경, 소수 Tune, 숫자가 아닌 Tune입니다.
  - 샘플 번호 모음: 중복 제거, Sampler가 아닌 악기 무시, `null` 건너뜀, 꺼진 층 건너뜀, 문자열 번호를 숫자로, 빈 입력입니다.
  - 노드 연결: 기록하는 가짜 컨텍스트로, `classic`의 정지 시각(노트 길이 + Release + 0.02초), Loop 구간, 음량(gain × velocity/127), `oneshot`의 정지 시각(샘플 길이 ÷ 속도), Loop와 Release 무시, Attack과 Decay 램프를 확인합니다.
- `docs/handoff/s1-11c_tests/render_sampler.mjs` (28건)는 실제 `OfflineAudioContext`로 소리를 그려서 숫자로 확인합니다.
  - A: 음높이가 440Hz, 880Hz, 220Hz로 나오고 Root와 Tune이 반영됩니다.
  - B: Release, Attack이 소리 크기에 반영됩니다.

- C: Loop가 꺼지면 샘플 길이(0.5초)에서 끝나고, 켜지면 노트가 끝날 때까지 이어지며 440Hz가 유지됩니다.
- D: `oneshot`은 노트가 아주 짧아도 샘플 끝까지 울리고, Loop, Sustain, Release를 무시하며, Decay로 꼬리가 줄고, 음높이는 따릅니다.
- E: 같은 샘플을 연속으로 치면 앞의 소리가 이어서 울려 합쳐집니다.
- F: gain 0.5는 피크가 절반, velocity 64는 64/127입니다.
- G: 샘플이 없거나 불러오지 못해도 오류 없이 조용합니다.
- H: Sampler와 Synth가 한 트랙에 있으면 둘 다 울리고, 같은 샘플은 한 번만 불러옵니다.